

COOPCORE

Manual tecnico de entrega final

Version	1.0 final
Fecha	17 de agosto de 2026
Repositorio	github.com/LLULIAN17/CoopCore
Esquema	SQL Server · CoopCoreDB · coop
Equipo	Equipo CoopCore

Documento consolidado de arquitectura, modelo de datos, seguridad, procedimientos, transacciones, optimizacion, API, instalacion y pruebas.

Control documental y contenido

Version	Fecha	Responsable	Cambio
1.0	2026-08-17	Equipo CoopCore	Consolidacion para entrega final.

Control documental y contenido	2
1. Resumen ejecutivo	4
La idea SaaS	4
2. Arquitectura de solucion	5
Responsabilidades	5
Principio rector	5
3. Dominio funcional	6
Reglas principales	6
4. Diagrama entidad-relacion final	7
5. Diccionario de datos	8
5.1 coop.Socio	9
5.2 coop.Rol	10
5.3 coop.Empleado	11
5.4 coop.ProductoFinanciero	12
5.5 coop.Cuenta	13
5.6 coop.Prestamo	14
5.7 coop.Cuota	15
5.8 coop.Movimiento	16
5.9 coop.Auditoria	17
5.10 coop.GestionCobranza	18
6. Objetos programables SQL	19
Funciones	19
Stored procedures	19
Parametros OUTPUT	19
Vistas y trigger	20
7. Transacciones explicitas y concurrencia	21
Patron transaccional	21
8. Control de acceso y seguridad	22
Defensa en profundidad	22
9. API REST .NET 10	23
10. Autenticacion, errores y uso del API	24
Flujo JWT	24

Contrato de errores	24
11. Analisis y optimizacion	25
Metodologia	25
Comparacion de costos estimados	25
11.1 Comparacion visual consolidada	26
11.2 Plan de movimientos	27
11.3 Plan de cuotas	28
11.4 Plan de auditoria	29
12. Instalacion reproducible	30
Requisitos	30
Orden de scripts	30
13. Puesta en marcha de API y frontend	31
API	31
Frontend	31
14. Estrategia de pruebas	32
Pruebas reversibles	32
15. Operacion, monitoreo y respaldo	33
Recuperacion	33
16. Guia de demostracion y defensa	34
Antes de exponer	34
17. Riesgos, limitaciones y trabajo futuro	35
18. Trazabilidad de requisitos	36
19. Glosario	37
20. Referencias internas y cierre	38
Conclusion	38

1. Resumen ejecutivo

CoopCore es una plataforma academica para una cooperativa de ahorro y credito. Su objetivo es demostrar que SQL Server puede concentrar reglas de negocio, seguridad, consistencia transaccional, auditoria y optimizacion, mientras una API REST y un frontend web actuan como canales de acceso controlados.

- 10 tablas normalizadas dentro del esquema funcional coop.
- 24 stored procedures con SET NOCOUNT ON, autor, fecha y manejo de errores.
- 2 funciones SQL utilizadas por procedimientos existentes.
- 10 procedimientos con transacciones explicitas y ROLLBACK seguro.
- 5 roles SQL y 5 logins de laboratorio con minimo privilegio.
- API .NET 10 con JWT, autorizacion por rol y errores consistentes.
- Frontend Next.js para cartera, morosidad, productos y cobranza.
- 5 indices evaluados con costos y planes XML antes/despues.

La idea SaaS

La propuesta SaaS permite que cooperativas pequenas operen una plataforma centralizada sin mantener logica financiera duplicada en cada cliente. La separacion por capas facilita desplegar la API y la interfaz de forma independiente, pero conserva una fuente unica de verdad en SQL Server. En una evolucion comercial, cada organizacion tendria aislamiento de datos, configuracion de productos, monitoreo, copias de seguridad y politicas de retencion propias.

2. Arquitectura de solucion



Responsabilidades

Capa	Responsabilidad	Tecnologia
Frontend	Interaccion, filtros, formularios y visualizacion.	Next.js / React / TypeScript
API	Autenticacion JWT, autorizacion, validacion minima y serializacion.	.NET 10 / ASP.NET Core / ADO.NET
Base de datos	Reglas, transacciones, permisos, auditoria y calculos.	SQL Server / T-SQL

Principio rector

La API no reemplaza las reglas financieras. Cada operacion relevante invoca un stored procedure del esquema coop. Esto reduce divergencias entre clientes, centraliza el control de concurrencia y permite auditar la operacion en la misma transaccion.

3. Dominio funcional

Modulo	Alcance
Autenticacion	Login, cambio de clave, bloqueo temporal y JWT.
Socios	Registro y consulta del asociado.
Cuentas	Apertura, saldo, deposito, retiro y transferencia.
Prestamos	Solicitud, aprobacion, rechazo, amortizacion y pago.
Morosidad	Busqueda, clasificacion y monto vencido.
Cartera	Indicadores, riesgo y proximos vencimientos.
Productos	Catalogo de ahorro, prestamos y certificados.
Cobranza	Alertas, compromisos y seguimiento.
Auditoria	Trazabilidad de acceso y operaciones.

Reglas principales

- Los saldos de cuenta nunca pueden ser negativos.
- Los movimientos contables son inmutables mediante trigger.
- Una transferencia registra salida y entrada con la misma referencia.
- Un pago no supera el pendiente de la cuota ni el saldo del prestamo.
- Las cuotas vencidas se calculan a una fecha de corte explicita.
- Cada escritura relevante genera un evento de auditoria.

4. Diagrama entidad-relacion final

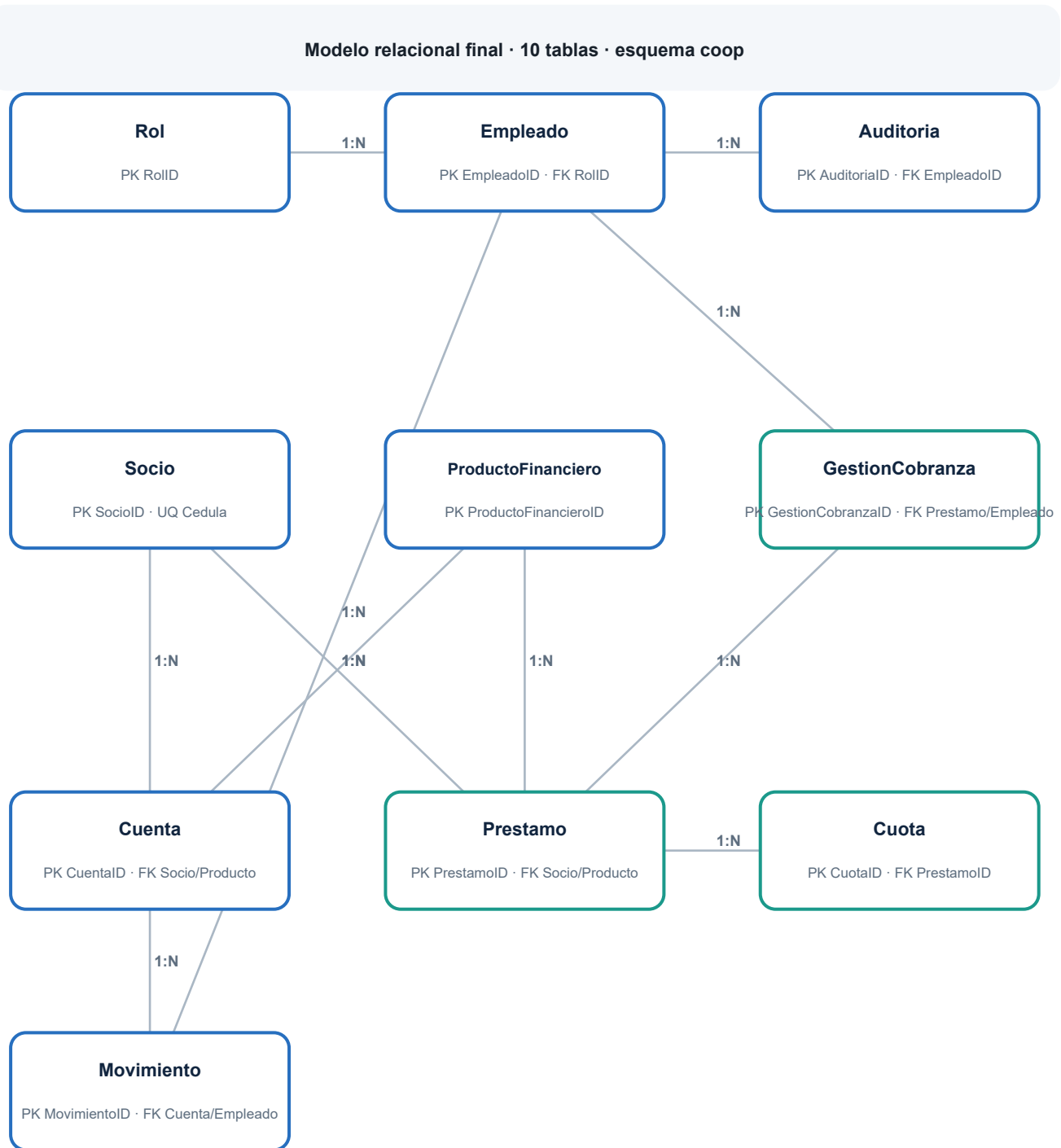


Figura 1. Modelo ER logico del esquema coop. Las relaciones son 1:N y se implementan mediante llaves foraneas.

Control de aprobacion docente: pendiente de validacion externa. El equipo debe presentar esta pagina al docente y registrar fecha o firma sin alterar el modelo aprobado.

5. Diccionario de datos

El diccionario se deriva de sql/01_schema_tables.sql y sql/15_alertas_cobranza.sql. Abreviaturas: PK llave primaria; FK llave foranea; UQ unicidad; CK validacion; DF valor predeterminado; NN no admite NULL.

Tabla	Columnas	Proposito
coop.Socio	9	Personas asociadas.
coop.Rol	5	Roles funcionales de aplicacion.
coop.Empleado	15	Operadores y credenciales.
coop.ProductoFinanciero	8	Catalogo de productos.
coop.Cuenta	8	Cuentas y saldos.
coop.Prestamo	11	Creditos y estado.
coop.Cuota	8	Calendario de pagos.
coop.Movimiento	8	Bitacora monetaria inmutable.
coop.Auditoria	9	Trazabilidad general.
coop.GestionCobranza	9	Seguimiento de cobro.

5.1 coop.Socio

Personas asociadas.

Columna	Tipo	Restricciones	Descripción
SocioID	INT IDENTITY	PK, NN	Identificador interno del socio.
Cedula	NVARCHAR(20)	UQ, NN	Identificador legal unico.
Nombre	NVARCHAR(80)	NN	Nombre del socio.
Apellido	NVARCHAR(80)	NN	Apellido del socio.
Correo	NVARCHAR(120)	NULL	Correo de contacto.
Telefono	NVARCHAR(30)	NULL	Telefono de contacto.
Direccion	NVARCHAR(250)	NULL	Direccion declarada.
Estado	NVARCHAR(20)	CK, DF	ACTIVO, INACTIVO o SUSPENDIDO.
FechaRegistro	DATETIME2	DF, NN	Fecha de alta del socio.

5.2 coop.Rol

Roles funcionales de aplicacion.

Columna	Tipo	Restricciones	Descripcion
RolID	INT IDENTITY	PK, NN	Identificador del rol de aplicacion.
NombreRol	NVARCHAR(50)	UQ, NN	Nombre unico del rol funcional.
Descripcion	NVARCHAR(200)	NULL	Alcance del rol.
Estado	NVARCHAR(20)	CK, DF	ACTIVO o INACTIVO.
FechaCreacion	DATETIME2	DF, NN	Fecha de creacion.

5.3 coop.Empleado

Operadores y credenciales.

Columna	Tipo	Restricciones	Descripción
EmpleadoID	INT IDENTITY	PK, NN	Identificador interno.
Cedula	NVARCHAR(20)	UQ, NN	Cedula unica del empleado.
Nombre	NVARCHAR(80)	NN	Nombre.
Apellido	NVARCHAR(80)	NN	Apellido.
Correo	NVARCHAR(120)	NULL	Correo institucional.
Telefono	NVARCHAR(30)	NULL	Telefono.
RolID	INT	FK, NULL	Rol funcional asignado.
Estado	NVARCHAR(20)	CK, DF	ACTIVO o INACTIVO.
FechaIngreso	DATETIME2	DF, NN	Fecha de ingreso.
NombreUsuario	NVARCHAR(50)	UQ filtrado	Usuario de la aplicacion.
PasswordHash	VARBINARY(64)	NULL	Hash SHA2_256 de la clave con salt.
PasswordSalt	VARBINARY(32)	NULL	Salt aleatorio de la clave.
UltimoLogin	DATETIME2	NULL	Ultimo acceso correcto.
IntentosFallidos	INT	DF, NN	Contador de accesos fallidos.
BloqueadoHasta	DATETIME2	NULL	Fin del bloqueo temporal.

5.4 coop.ProductoFinanciero

Catalogo de productos.

Columna	Tipo	Restricciones	Descripcion
ProductoFinancieroID	INT IDENTITY	PK, NN	Identificador del producto.
CodigoProducto	NVARCHAR(20)	UQ, NN	Codigo comercial unico.
NombreProducto	NVARCHAR(100)	UQ, NN	Nombre comercial unico.
TipoProducto	NVARCHAR(30)	CK, NN	AHORRO, PRESTAMO, CERTIFICADO u OTRO.
TasaInteres	DECIMAL(18,2)	CK, DF	Tasa porcentual entre 0 y 100.
MontoMinimoApertura	DECIMAL(18,2)	CK, DF	Monto minimo no negativo.
Estado	NVARCHAR(20)	CK, DF	ACTIVO o INACTIVO.
FechaCreacion	DATETIME2	DF, NN	Fecha de alta.

5.5 coop.Cuenta

Cuentas y saldos.

Columna	Tipo	Restricciones	Descripción
CuentaID	INT IDENTITY	PK, NN	Identificador interno.
NumeroCuenta	NVARCHAR(30)	UQ, NN	Numero publico unico.
SocioID	INT	FK, NN	Socio propietario.
ProductoFinancieroID	INT	FK, NN	Producto asociado.
CreadaPorEmpleadoID	INT	FK, NULL	Empleado que abrio la cuenta.
Saldo	DECIMAL(18,2)	CK, DF	Saldo disponible no negativo.
EstadoCuenta	NVARCHAR(20)	CK, DF	ACTIVA, INACTIVA, BLOQUEADA o CERRADA.
FechaApertura	DATETIME2	DF, NN	Fecha de apertura.

5.6 coop.Prestamo

Creditos y estado.

Columna	Tipo	Restricciones	Descripcion
PrestamoID	INT IDENTITY	PK, NN	Identificador interno.
NumeroPrestamo	NVARCHAR(30)	UQ, NN	Numero publico unico.
SocioID	INT	FK, NN	Socio deudor.
ProductoFinancieroID	INT	FK, NN	Producto de credito.
AprobadoPorEmpleadoID	INT	FK, NULL	Empleado aprobador.
MontoOriginal	DECIMAL(18,2)	CK, NN	Principal otorgado.
SaldoPendiente	DECIMAL(18,2)	CK, NN	Saldo entre cero y monto original.
TasaInteres	DECIMAL(18,2)	CK, NN	Tasa contractual.
PlazoMeses	INT	CK, NN	Plazo positivo en meses.
FechaDesembolso	DATETIME2	DF, NN	Fecha de desembolso o aprobacion.
EstadoPrestamo	NVARCHAR(20)	CK, DF	SOLICITADO, ACTIVO, PAGADO, MORA o CANCELADO.

5.7 coop.Cuota

Calendario de pagos.

Columna	Tipo	Restricciones	Descripción
CuotaID	INT IDENTITY	PK, NN	Identificador de cuota.
PrestamoID	INT	FK, NN	Prestamo propietario.
NumeroCuota	INT	UQ compuesta, CK	Secuencia positiva dentro del prestamo.
FechaVencimiento	DATE	NN	Fecha limite de pago.
MontoCuota	DECIMAL(18,2)	CK, NN	Monto contractual positivo.
MontoPagado	DECIMAL(18,2)	CK, DF	Acumulado entre cero y monto de cuota.
FechaPago	DATETIME2	NULL	Fecha de pago completo.
EstadoCuota	NVARCHAR(20)	CK, DF	PENDIENTE, PAGADA, VENCIDA o PARCIAL.

5.8 coop.Movimiento

Bitacora monetaria inmutable.

Columna	Tipo	Restricciones	Descripcion
MovimientoID	BIGINT IDENTITY	PK, NN	Identificador contable.
CuentaID	INT	FK, NN	Cuenta afectada.
TipoMovimiento	NVARCHAR(30)	CK, NN	Deposito, retiro, transferencia o ajuste.
Monto	DECIMAL(18,2)	CK, NN	Monto positivo.
Referencia	NVARCHAR(50)	NULL	Referencia compartida de operacion.
Observacion	NVARCHAR(300)	NULL	Detalle operativo.
FechaMovimiento	DATETIME2	DF, NN	Marca temporal.
EjecutadoPorEmpleadoID	INT	FK, NN	Empleado responsable.

5.9 coop.Auditoria

Trazabilidad general.

Columna	Tipo	Restricciones	Descripcion
AuditorialID	BIGINT IDENTITY	PK, NN	Identificador del evento.
Entidad	NVARCHAR(100)	NN	Objeto de negocio afectado.
EntidadID	NVARCHAR(100)	NULL	Identificador textual del objeto.
Accion	NVARCHAR(30)	CK, NN	INSERT, UPDATE, DELETE, EXECUTE, LOGIN u OTRO.
Descripcion	NVARCHAR(4000)	NULL	Resumen del evento.
FechaEvento	DATETIME2	DF, NN	Marca temporal.
UsuarioSQL	NVARCHAR(128)	DF, NN	Login original de SQL Server.
UsuarioBD	NVARCHAR(128)	DF, NN	Usuario de base de datos.
EmpleadoID	INT	FK, NULL	Empleado de aplicacion asociado.

5.10 coop.GestionCobranza

Seguimiento de cobro.

Columna	Tipo	Restricciones	Descripción
GestionCobranzaID	BIGINT IDENTITY	PK, NN	Identificador de gestion.
PrestamoID	INT	FK, NN	Prestamo gestionado.
EmpleadoID	INT	FK, NN	Empleado responsable.
FechaGestion	DATETIME2	DF, NN	Marca temporal.
TipoGestion	NVARCHAR(20)	CK, NN	Llamada, correo, SMS, visita, acuerdo u otro.
Resultado	NVARCHAR(30)	CK, NN	Resultado normalizado del contacto.
Comentario	NVARCHAR(500)	NN	Detalle obligatorio.
FechaCompromiso	DATE	NULL	Fecha prometida de pago.
MontoCompromiso	DECIMAL(18,2)	CK, NULL	Monto positivo prometido.

6. Objetos programables SQL

Funciones

Funcion	Tip	Usos real
coop.fn_CalcularMoraCuota	Escalar	sp_ConsultarAlertasCobranza y funcion tabular.
coop.fn_ObtenerCuotasVencidas	Tabla en linea	sp_BuscarClientesMorosos y sp_ConsultarDashboardCartera.

Stored procedures

Procedimiento	Responsabilidad
sp_ValidarLogin	Autentica, audita intentos y aplica bloqueo temporal.
sp_ObtenerUsuarioPorCredenciales	Valida credenciales para uso interno.
sp_CambiarPassword	Renueva salt y hash de la clave.
sp_ConsultarSocio	Consulta socio y resumen financiero.
sp_ConsultarSaldo	Consulta saldo y ultimo movimiento.
sp_ConsultarMovimientos	Lista movimientos por rango.
sp_RegistrarSocio	Crea un socio y audita.
sp_CrearCuenta	Abre cuenta y deposito inicial opcional.
sp_ConsultarPrestamo	Devuelve resumen y cuotas.
sp_ConsultarAuditoria	Filtra eventos de auditoria.
sp_RegistrarDeposito	Deposito transaccional con OUTPUT.
sp_RegistrarRetiro	Retiro con bloqueo y saldo suficiente.
sp_RegistrarTransferencia	Transferencia atomica con dos OUTPUT.
sp_PagarCuota	Pago de cuota, cuenta y prestamo atomicos.
sp_SolicitarPrestamo	Crea solicitud con identificadores OUTPUT.
sp_AprobarPrestamo	Activa una solicitud.
sp_RechazarPrestamo	Cancela solicitud con motivo.
sp_GenerarAmortizacion	Genera cuotas y ajusta redondeo.
sp_BuscarClientesMorosos	Busca y pagina cartera vencida.
sp_ConsultarDashboardCartera	Calcula indicadores y riesgo.
sp_BuscarProductosFinancieros	Consulta catalogo y uso.
sp_GuardarProductoFinanciero	Crea o actualiza productos.
sp_ConsultarAlertasCobranza	Prioriza cuotas para cobro.
sp_RegistrarGestionCobranza	Registra seguimiento y auditoria.

Parametros OUTPUT

- sp_RegistrarDeposito: @NuevoSaldo y @NuevoMovimientoID.
- sp_RegistrarTransferencia: @MovimientoSalidaID y @MovimientoEntradaID.
- sp_SolicitarPrestamo: @NuevoPrestamoID y @NuevoNumeroPrestamo.

- Los parametros tienen valor predeterminado NULL y conservan los result sets consumidos por la API.

Vistas y trigger

Objeto	Tipo	Responsabilidad
coop.vw_CuentasResumen	Vista	Cuenta, socio, producto y saldo.
coop.vw_MovimientosAuditoria	Vista	Movimiento con cuenta, socio y empleado.
coop.vw_PrestamosResumen	Vista	Prestamo con agregados de cuotas.
coop.vw_SociosConsulta	Vista	Socio con cuentas y cartera consolidada.
coop.tr_Movimiento_Inmutable	Trigger	Impide UPDATE y DELETE contable.

Todos los procedimientos comienzan con SET NOCOUNT ON. Las operaciones criticas usan TRY/CATCH y devuelven result sets, codigos de estado o parametros OUTPUT segun corresponda.

7. Transacciones explícitas y concurrencia

Patron transaccional

Los procedimientos monetarios activan SET XACT_ABORT ON, abren una transacción dentro de BEGIN TRY, validan filas bloqueadas con UPDLOCK y HOLDLOCK, escriben las tablas de negocio y auditoría, y solo entonces confirman. BEGIN CATCH revierte cuando @@TRANCOUNT es mayor que cero y relanza un error de dominio.

```
SET XACT_ABORT ON;
BEGIN TRY
BEGIN TRANSACTION;
-- validaciones, bloqueos, escritura y auditoria
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
THROW;
END CATCH;
```

Procedimiento	Tablas atomizadas	Control especial
Deposito	Cuenta, Movimiento, Auditoria	UPDLOCK/HOLDLOCK
Retiro	Cuenta, Movimiento, Auditoria	Saldo suficiente
Transferencia	2 Cuenta, 2 Movimiento, Auditoria	Orden estable de bloqueos
Pago de cuota	Cuenta, Cuota, Prestamo, Movimiento, Auditoria	Limites de pago
Solicitud	Prestamo, Auditoria	Numero unico
Aprobacion/Rechazo	Prestamo, Auditoria	Estado esperado
Amortizacion	Cuota, Auditoria	Sin cuotas previas
Producto/Cobranza	Objeto, Auditoria	TRY/CATCH y rollback

8. Control de acceso y seguridad

Rol SP	Perfil	Acceso principal
rol_admin_coop	Administrador	Todos los SPs operativos.
rol_cajero_coop	Caja	Socios, cuentas, movimientos y pagos.
rol_oficial_credito_coop	Credito	Prestamos, mora, cartera y cobranza.
rol_auditor_coop	Auditoria	Consultas y vistas; escritura denegada.
rol_api_coop	Servicio	Solo SPs usados por la API; sin tablas directas.

Defensa en profundidad

- Logins y usuarios separados por perfil.
- Permisos EXECUTE por objeto y DENY de escritura directa.
- Passwords de aplicacion almacenadas con salt y SHA2_256.
- Bloqueo temporal despues de cinco intentos fallidos.
- JWT con vigencia de 60 minutos y rol de aplicacion.
- La cadena de conexion real permanece fuera del repositorio.
- Las credenciales seed son exclusivamente academicas.

9. API REST .NET 10

La API sigue Controllers -> Interfaces -> Services -> Db. SqlExecutor crea SqlCommand con CommandType.StoredProcedure y transforma filas a modelos de respuesta. Los controladores aplican autorización por rol y producen ApiResponse uniforme.

Método	Ruta	Rol	Stored procedure
POST	/api/auth/login	Publico	sp_ValidarLogin
POST	/api/auth/cambiar-password	Autenticado	sp_CambiarPassword
GET	/api/socios/{id}	Admin/Cajero	sp_ConsultarSocio
POST	/api/socios	Admin/Cajero	sp_RegistrarSocio
POST	/api/cuentas	Admin/Cajero	sp_CrearCuenta
GET	/api/cuentas/{numero}/saldo	Admin/Cajero	sp_ConsultarSaldo
GET	/api/cuentas/{numero}/movimientos	Admin/Cajero	sp_ConsultarMovimientos
POST	/api/cuentas/depositos	Admin/Cajero	sp_RegistrarDeposito
POST	/api/cuentas/retiros	Admin/Cajero	sp_RegistrarRetiro
POST	/api/cuentas/transferencias	Admin/Cajero	sp_RegistrarTransferencia
GET	/api/prestamos/{numero}	Admin/Oficial	sp_ConsultarPrestamo
POST	/api/prestamos	Admin/Oficial	sp_SolicitarPrestamo
POST	/api/prestamos/{numero}/aprobar	Admin/Oficial	sp_AprobarPrestamo
POST	/api/prestamos/{numero}/rechazar	Admin/Oficial	sp_RechazarPrestamo
POST	/api/prestamos/{numero}/amortizacion	Admin/Oficial	sp_GenerarAmortizacion
POST	/api/prestamos/{numero}/cuotas/{n}/pagos	Admin/Cajero	sp_PagarCuota
GET	/api/clientes-morosos	Admin/Oficial	sp_BuscarClientesMorosos
GET	/api/cartera/dashboard	Admin/Oficial	sp_ConsultarDashboardCartera
GET	/api/productos-financieros	Autenticado	sp_BuscarProductosFinancieros
POST/PUT	/api/productos-financieros	Admin	sp_GuardarProductoFinanciero
GET	/api/cobranza/alertas	Admin/Oficial	sp_ConsultarAlertasCobranza
POST	/api/cobranza/gestiones	Admin/Oficial	sp_RegistrarGestionCobranza
GET	/api/auditoria	Admin/Auditor	sp_ConsultarAuditoria

10. Autenticacion, errores y uso del API

Flujo JWT

- El cliente envia usuario y password a POST /api/auth/login.
- sp_ValidarLogin verifica hash, estado y bloqueo y registra auditoria.
- La API emite un JWT firmado con usuario, empleado y rol.
- El cliente envia Authorization: Bearer TOKEN en cada ruta protegida.
- La politica de ASP.NET Core devuelve 401 sin identidad y 403 sin rol.

```
curl.exe -X POST http://localhost:5000/api/auth/login `
-H "Content-Type: application/json" `
-d '{"usuario":"mlrojas","password":"Lab_Cajero_001"}'

$token = "<datos.token>"
curl.exe http://localhost:5000/api/cuentas/CTA-10001/saldo `
-H "Authorization: Bearer $token"
```

Contrato de errores

HTTP	Situacion
400	Validacion o regla de negocio.
401	Token ausente, invalido o vencido.
403	Rol insuficiente.
404	Recurso no encontrado.
500	Error no controlado, sin exponer secretos.

11. Analisis y optimizacion

Metodologia

- Linea base con SET STATISTICS IO/TIME antes de crear indices.
- Cinco indices vinculados a patrones reales de acceso.
- Planes XML estimados antes y despues sobre la misma instancia y seed.
- DROP/CREATE de cada indice dentro de una transaccion que se revierte.
- Comparacion de costo, operadores, indice elegido y lecturas logicas.

Comparacion de costos estimados

Indice	Costo antes	Costo despues	Diferencia
Movimiento_Cuenta_Fecha	0.01467280	0.00328680	-77.60%
Cuenta_Socio_Saldo	0.00328942	0.00328420	-0.16%
Prestamo_Socio_Saldo	0.00328942	0.00328420	-0.16%
Cuota_Prestamo_Numero	0.00730559	0.00328640	-55.02%
Auditoria_Accion_Fecha	0.01474430	0.00330000	-77.62%

11.1 Comparacion visual consolidada

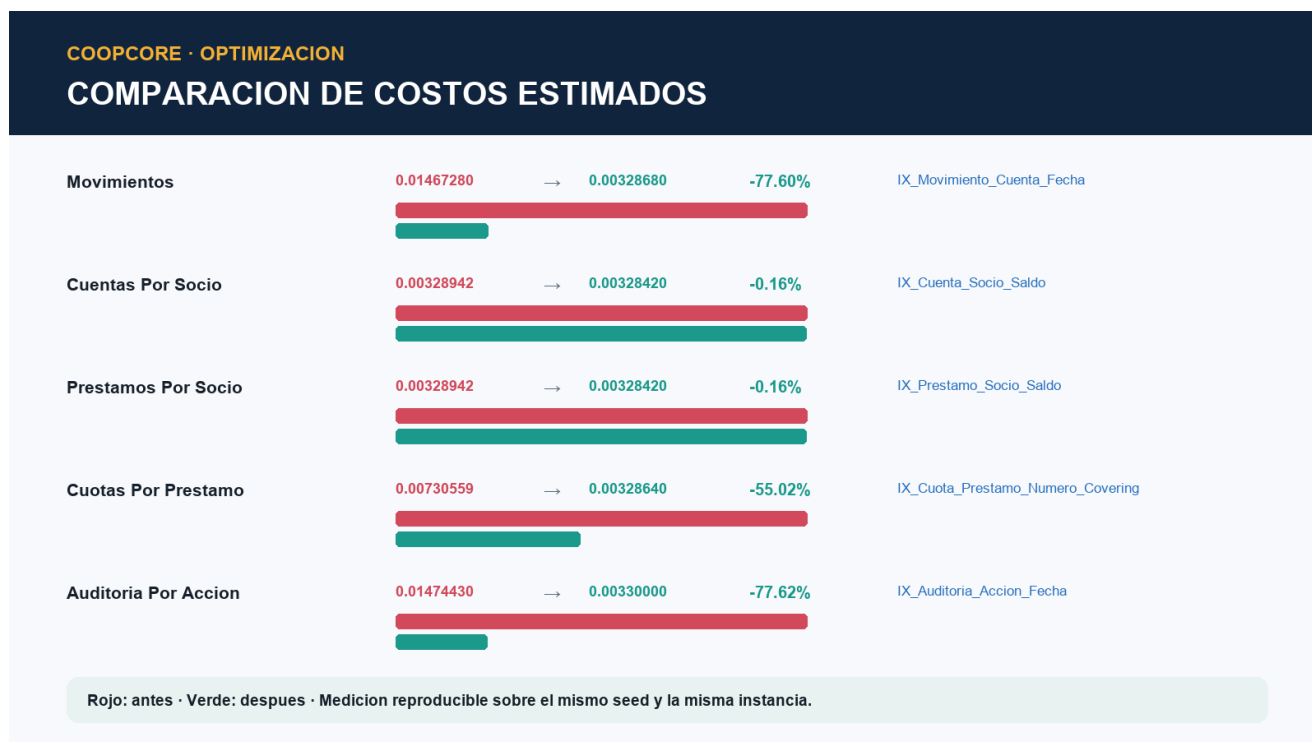
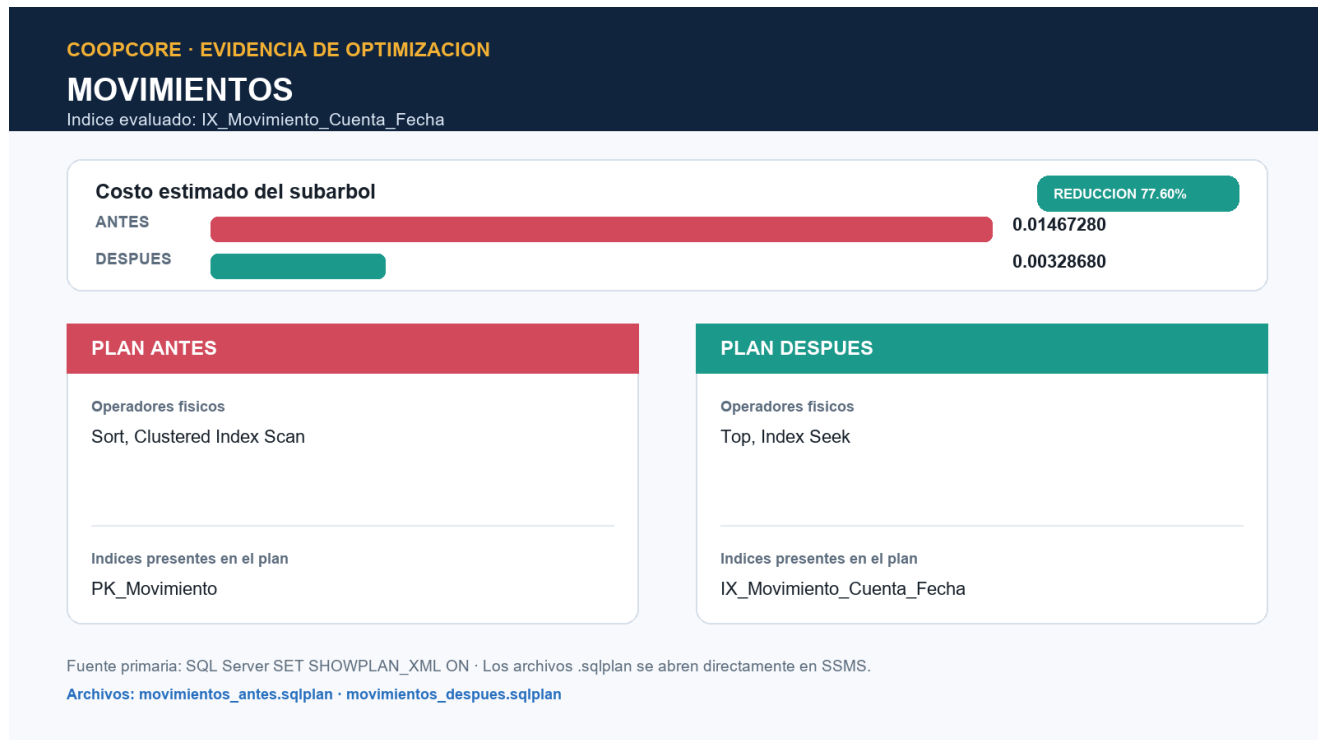


Figura 2. Costo estimado antes y despues. Fuente: planes SHOWPLAN XML versionados en docs/evidencias/planes.

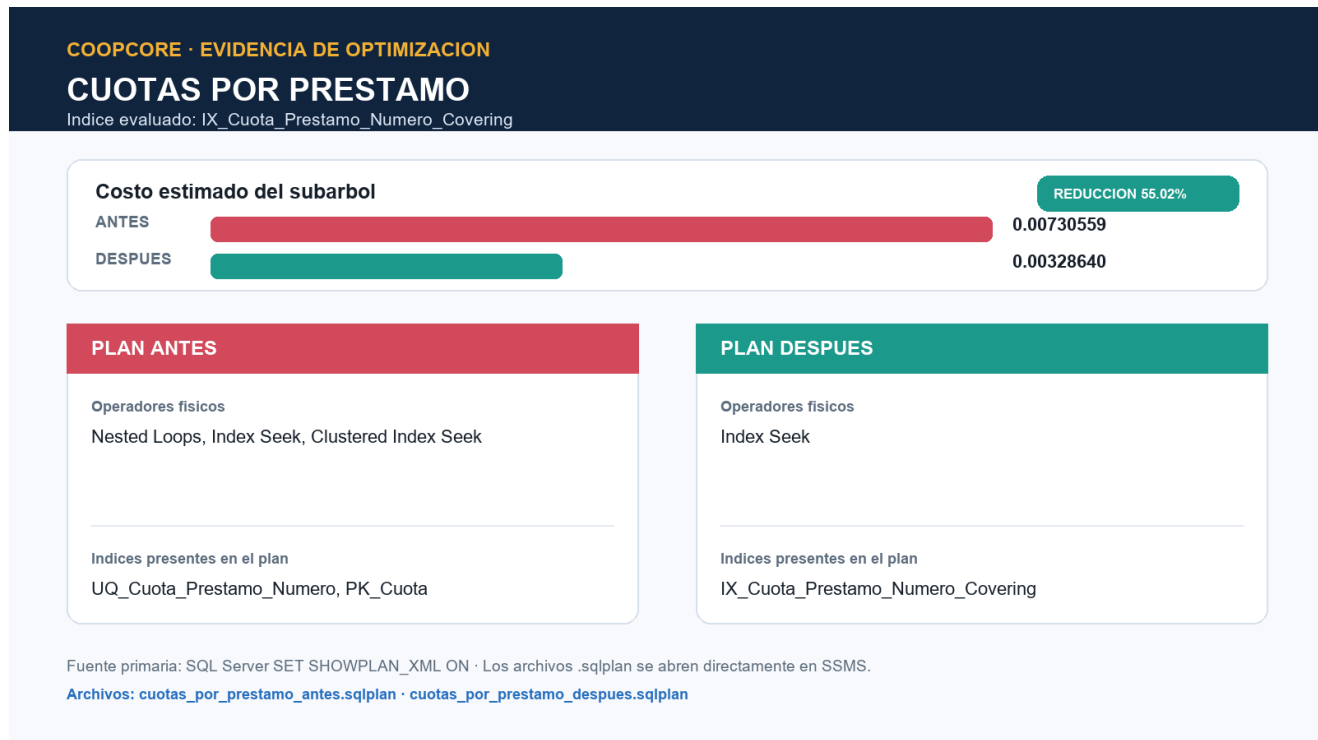
Los indices de Cuenta y Prestamo muestran una mejora pequena por el volumen reducido del seed, aunque cambian de Clustered Index Scan a Index Seek. Cuota reduce lecturas de 12 a 4. Movimiento y Auditoria eliminan Sort y usan el indice diseñado.

11.2 Plan de movimientos



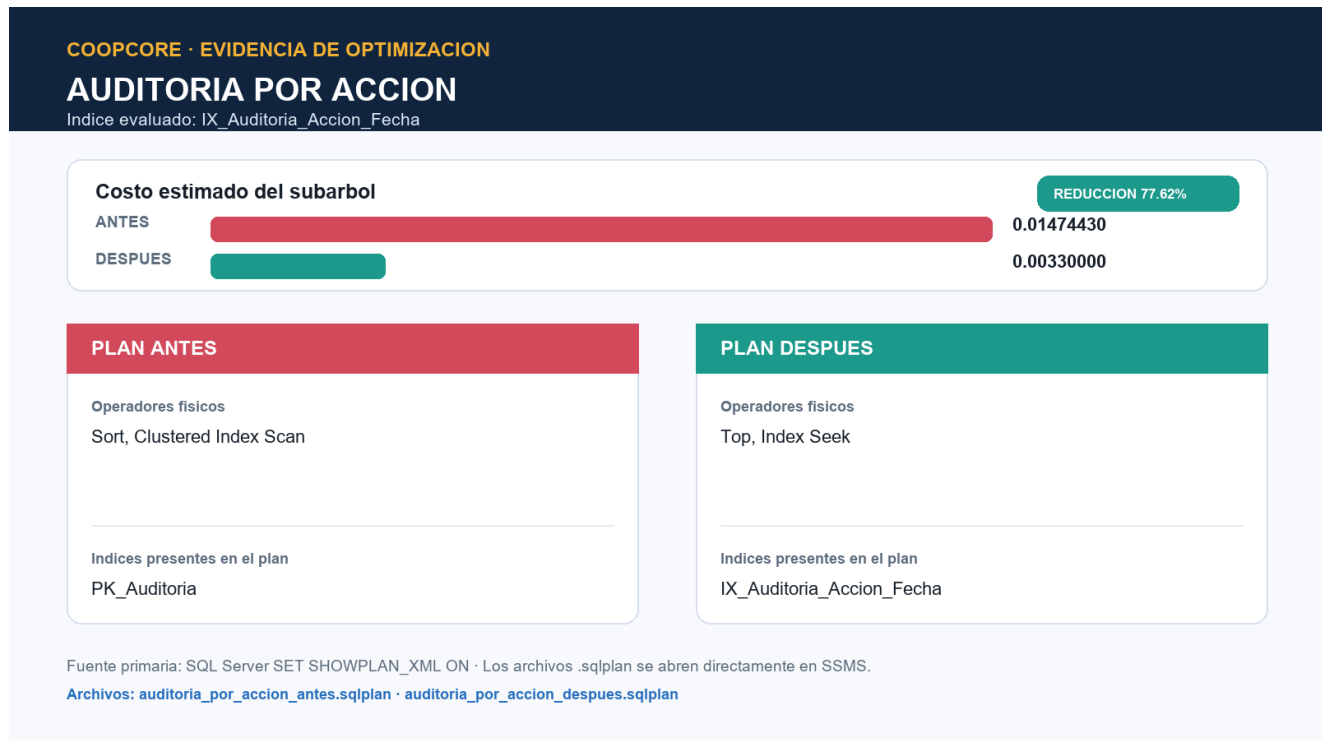
Visualizacion derivada del archivo .sqlplan original. El plan completo puede abrirse en SSMS.

11.3 Plan de cuotas



Visualizacion derivada del archivo .sqlplan original. El plan completo puede abrirse en SSMS.

11.4 Plan de auditoria



Visualizacion derivada del archivo .sqlplan original. El plan completo puede abrirse en SSMS.

12. Instalacion reproducible

Requisitos

- SQL Server Express o superior.
- sqlcmd con ODBC Driver 18.
- .NET SDK 10.
- Node.js y npm para el frontend.
- Git y GitHub CLI para colaboracion.

Orden de scripts

#	Script
1	00_create_database.sql
2	01_schema_tables.sql
3	02_seed_data.sql
4	03_functions.sql
5	03_views.sql
6	04_stored_procedures.sql
7	05_transactions.sql
8	06_security.sql
9	07_security_tests.sql
10	08_concurrency_tests.sql
11	09_execution_plan_baseline.sql
12	09_indexes_optimization.sql
13	10_revision3_tests.sql
14	11_busqueda_clientes_morosos.sql
15	12_busqueda_clientes_morosos_tests.sql
16	13_dashboard_cartera.sql
17	14_productos_financieros.sql
18	15_alertas_cobranza.sql
19	16_ampliacion_50_tests.sql
20	17_entrega_final_tests.sql

13. Puesta en marcha de API y frontend

API

```
Copy-Item api\coopcore-api\coopcore-api\appsettings.example.json `
api\coopcore-api\coopcore-api\appsettings.Development.json

dotnet restore api\coopcore-api\coopcore-api\coopcore-api.csproj
dotnet build api\coopcore-api\coopcore-api\coopcore-api.csproj
dotnet run --project api\coopcore-api\coopcore-api\coopcore-api.csproj `
--urls http://localhost:5000
```

Frontend

```
Set-Location frontend\app
npm ci
npm run lint
npm test
npm run dev
```

Para demostrar integracion real, el frontend debe usar la URL de la API y un JWT obtenido por login. Los datos demo sirven como degradacion visual, pero no sustituyen la prueba en vivo con SQL Server.

14. Estrategia de pruebas

Nivel	Prueba	Criterio
SQL instalacion	Secuencia 00 a 17 sobre base nueva	Todos los scripts terminan con codigo 0.
SQL funcional	Funciones y OUTPUT	Valores esperados y rollback.
Seguridad	Roles, grants y denies	Acceso permitido/denegado segun perfil.
Concurrencia	Bloqueos y transacciones	Sin saldo negativo ni escrituras parciales.
API compilacion	dotnet build Release	0 errores y 0 advertencias.
API HTTP	Login, lecturas, escrituras y roles	200/201, 400, 401 y 403 esperados.
Frontend	lint, build y render	Sin errores y HTML verificable.
PDF	Render completo	20+ paginas sin recortes ni solapamientos.

Pruebas reversibles

Los scripts de revision y entrega final encapsulan operaciones de escritura en transacciones externas y ejecutan ROLLBACK. Esto permite validar identificadores, saldos y auditoria sin contaminar el seed ni dificultar ejecuciones posteriores.

15. Operacion, monitoreo y respaldo

- Monitorear crecimiento de Movimiento, Auditoria, Cuota y GestionCobranza.
- Revisar fragmentacion y estadísticas de índices en ventanas planificadas.
- Respalidar CoopCoreDB antes de cambios de esquema y comprobar restauracion.
- Rotar la clave JWT y las credenciales SQL fuera del código fuente.
- Consultar errores de la API sin registrar passwords ni tokens completos.
- Definir retencion de auditoria conforme a politica institucional.

Recuperacion

Ante una falla se debe detener la escritura, conservar logs, restaurar el ultimo respaldo valido en una instancia aislada, aplicar respaldos diferenciales o de log si existen, ejecutar pruebas de integridad y solo despues redirigir la API. El procedimiento exacto depende del modelo de recuperacion configurado por la organizacion.

16. Guia de demostracion y defensa

Minuto	Acción	Evidencia
0-2	Presentar arquitectura y ER.	SQL concentra la logica.
2-4	Mostrar login y JWT real.	Auditoria y expiracion.
4-7	Deposito o transferencia.	Saldo, movimientos y OUTPUT.
7-10	Prestamo y amortizacion.	Transaccion y cuotas.
10-12	Morosidad, cartera y cobranza.	Funciones SQL usadas.
12-14	Seguridad por rol.	403 y permisos SQL.
14-16	Planes antes/despues.	Costo y Index Seek.
16-18	Preguntas del equipo.	Cada integrante explica cualquier capa.

Antes de exponer

- SQL Server iniciado y base recién validada.
- API en localhost:5000 y health 200.
- Frontend conectado a la API, no solo a datos demo.
- Usuarios de cada rol probados.
- Planes .sqlplan listos para abrir en SSMS.
- Cada integrante conoce funciones, OUTPUT, transacciones, seguridad y API.

17. Riesgos, limitaciones y trabajo futuro

Riesgo o limit.	Tratamiento actual	Exolución
Seed pequeno	Comparar operadores y costo ademas de tiempo.	Pruebas con volumen sintetico.
SQL Express con autenticacion Windows	API local usa conexion integrada para pruebas.	Habilitar modo mixto en despliegue controlado.
Datos demo del frontend	Modo degradado identificado.	Selector explicito API/demo y prueba E2E.
JWT simetrico	Clave fuera del repositorio.	Secret manager y rotacion.
Sin multi-tenancy	Modelo para una cooperativa.	TenantID y aislamiento si se comercializa.
Aprobacion ER externa	Diagrama final preparado.	Registrar aprobacion docente.

18. Trazabilidad de requisitos

Requisito	Implementación	Evidencia
2 funciones usadas	03_functions + SPs de mora/cartera	17_entrega_final_tests.sql
Parametros entrada/salida	3 SPs con OUTPUT opcional	Pruebas 5, 6 y 7
Autor y fecha	24 cabeceras normalizadas	Scripts 04, 05, 11, 13, 14 y 15
Optimizacion numerica	5 costos antes/despues	docs/evidencias/planes
Diagrama ER	10 tablas y relaciones	Seccion 4
Diccionario	Todas las columnas y restricciones	Seccion 5
Manual 20+ paginas	PDF consolidado	Este documento
Instalacion limpia	Secuencia automatizada	Evidencia sqlcmd
Demo real	Checklist SQL + API + JWT + frontend	Seccion 16

19. Glosario

Termino	Definicion
Atomicidad	Una transaccion confirma todas sus escrituras o ninguna.
Costo estimado	Valor relativo calculado por el optimizador para un plan.
Index Seek	Acceso dirigido a un rango de claves de un indice.
JWT	Token firmado que transporta identidad y claims.
Mora	Atraso de una cuota respecto de la fecha de corte.
OUTPUT	Parametro mediante el cual un SP devuelve un valor al llamador.
Ownership chaining	Resolucion de permisos entre objetos con propietario comun.
Result set	Conjunto tabular devuelto por una consulta o procedimiento.
Rollback	Reversion de cambios no confirmados.
Salt	Valor aleatorio agregado antes de calcular un hash de password.
Stored procedure	Programa T-SQL almacenado y ejecutado en la base.

20. Referencias internas y cierre

- README.md - instalacion, modulos y endpoints.
- sql/01_schema_tables.sql - modelo fisico.
- sql/03_functions.sql - funciones de dominio.
- sql/04_stored_procedures.sql y sql/05_transactions.sql - operaciones principales.
- sql/06_security.sql - logins, usuarios, roles y permisos.
- docs/optimizacion_indices.md - comparacion cuantitativa.
- docs/evidencias/planes - planes XML, CSV e imagenes.
- api/coopcore-api/README.md - configuracion y contrato HTTP.
- docs/evidencias/sistema_completo_smoke_test.md - validacion integral.

Conclusion

CoopCore cumple el objetivo academico de mantener la logica de negocio en SQL Server y exponerla de forma segura mediante una API delgada. La entrega final incorpora las funciones requeridas, parametros OUTPUT, cabeceras trazables, comparacion numerica de optimizacion, modelo ER, diccionario de datos y una guia reproducible de instalacion, prueba y defensa.

Fin del manual tecnico · Version 1.0